Cloud-Based Online Examination System

Project Objective

The objective of this project is to provide a secure, cloud-based platform for conducting online examinations. It allows administrators to create exams and students to take them remotely while ensuring automatic evaluation, secure authentication, centralized data storage, and cloud accessibility.

Problem Statement

Traditional examination systems have several drawbacks:

- Paper-based exams require manual evaluation.
- Results take time.
- Data is stored locally, making it difficult to access remotely.
- There is no centralized management.
- Difficult to scale for multiple users.

My project solves these issues by moving the application and database to the cloud.

Technologies Used

Frontend

- HTML5
- CSS3
- Bootstrap 5
- JavaScript

Purpose:

- Responsive UI
- Interactive pages
- Timer
- Validation

Backend

Python

Framework:

Flask

Purpose:

- Handles routing
- Authentication
- Business logic
- Session management

Database

TiDB Cloud

Why TiDB?

- MySQL compatible
- Managed cloud database
- Highly available
- Scalable
- Free tier available

ORM

Flask SQLAlchemy

Purpose:

Instead of writing SQL everywhere, models are mapped directly to database tables.

Example:

```
User
Exam
Question
Result
```

Authentication

Flask Login

Purpose:

- Secure login
- Session management
- User authentication

Database Driver

PyMySQL

Purpose:

Connects Flask to TiDB Cloud.

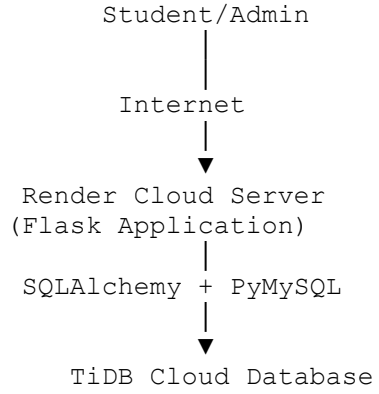
Deployment

Render

Purpose:

Hosts the Flask application online.

Cloud Architecture



Project Workflow

Step 1

User opens the website.

```
Browser
↓
```

```
Home Page
```

Step 2

User registers.

Information is stored in TiDB Cloud.

```
User
↓
Register Form
↓
TiDB Cloud
```

Step 3

User logs in.

Flask checks

```
Email
Password
```

against the cloud database.

If correct,

Session is created.

Step 4

Admin Dashboard

Admin can

- Create Exam
- Add Questions
- View Results

Step 5

Student Dashboard

Student

- Views available exams
- Starts exam

Step 6

Exam Starts

Features include

- Timer
- Auto submission
- Tab switching detection

Step 7

Evaluation

When student submits

Flask

↓

Calculates

- Score
- Percentage
- Pass/Fail

↓

Stores result

↓

Shows score instantly

Database Tables

You have **5 tables**

users

Stores

- Admin
- Student

Columns

`id`

`name`

`email`

`password_hash`

`role`

`created_at`

---

exams

Stores

`Exam Title`

`Subject`

`Duration`

`Pass Percentage`

---

questions

Stores

- Question
- Options
- Correct Answer
- Marks

---

student_answers

Stores

Student's selected options.

---

results

Stores

`Score`

`Percentage`

`Pass/Fail`

`Submission Time`

Why Flask?

Flask is lightweight, modular, easy to understand, and suitable for developing scalable web applications with RESTful architecture.

Why SQLAlchemy?

Instead of writing SQL queries,

we define models.

Example

```
class User(db.Model):
```

SQLAlchemy automatically converts this into SQL queries.

Why TiDB Cloud?

Earlier

Database was

`localhost`

Only my laptop could access it.

Now

Database is

```
Cloud
```

Anyone from anywhere can access it through the application.

Advantages

- Centralized
- Highly Available
- Automatic Backup
- Scalable
- Managed Service

Why Render?

Instead of running

```
python app.py
```

on your laptop,

Render hosts

```
Flask
```

```
Gunicorn
```

```
Python
```

24×7 on the cloud.

Anyone with the URL can access the project.

What did i change during deployment?

Originally

```
Flask
```

```
↓
```

```
localhost MySQL
```

After deployment

```
Flask

↓

TiDB Cloud
```

Changes made

Earlier

```
DB_HOST=localhost

DB_PORT=3306
```

Now

```
DB_HOST=gateway...

DB_PORT=4000
```

I also

- Generated database password
- Enabled SSL connection
- Added environment variables on Render

Environment Variables

Instead of storing passwords in code

we used

```
.env
```

Variables

```
SECRET_KEY

DB_HOST

DB_PORT

DB_USER

DB_PASSWORD

DB_NAME
```

Benefits

- Secure

- Easy deployment
- Reusable

GitHub

We pushed the complete source code to GitHub.

Render automatically pulls code from GitHub whenever new changes are pushed.

Workflow

```
VS Code

↓

Git

↓

GitHub

↓

Render

↓

Live Website
```

Deployment Process

1. Developed locally.

↓

2. Tested locally.

↓

3. Created TiDB Cloud database.

↓

4. Exported local database.

↓

5. Imported into TiDB Cloud.

↓

6. Updated Flask configuration.

↓

7. Pushed project to GitHub.

↓

8. Connected GitHub with Render.

↓

9. Added Environment Variables.

↓

10. Render deployed automatically.

Security Features

- Password Hashing
- Session Authentication
- Cloud Database
- Environment Variables
- Auto Logout
- Tab Switching Detection

Advantages

- Cloud Hosted
- Automatic Evaluation
- Centralized Database
- Responsive Design
- Secure Authentication
- Easy to Scale
- Accessible Anywhere

Future Enhancements

- Negative Marking

- Fill in the Blanks
- True/False Questions
- AI Question Generation
- Excel Question Import
- Question Bank
- PDF Reports
- Certificate Generation
- Analytics Dashboard
- Email Notifications